PROBLEM 5 · SOLUTION

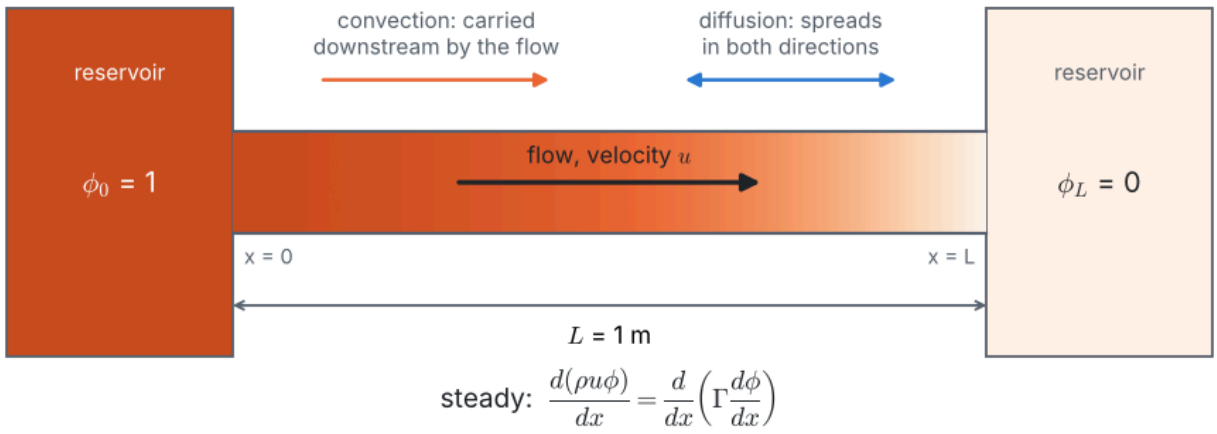
Steady convection and diffusion in one dimension

Step-by-step solution with the Finite Difference Method

Plan. We follow the 7 pillars of CFD: geometry → grid → physics → numerical scheme → solution setup → compute → post-processing. All code below is plain Python (lists and loops only; `math` for the exact solution and the order of accuracy). It is the code of the notebook `03_fdm_solver.ipynb`, where it can be run cell by cell.

Step 1 · Geometry and data

A fluid flows with constant velocity u through a channel of length $L = 1$ m, from a reservoir with $\phi_0 = 1$ to a reservoir with $\phi_L = 0$. The density is $\rho = 1$ kg/m³ and the diffusion coefficient $\Gamma = 0.1$ kg/(m·s). We study a slow flow, $u = 0.1$ m/s, and a fast flow, $u = 2.5$ m/s.



The channel between the two reservoirs.

```
# ---- domain and fluid ----------------------------------------------------
L = 1.0           # length of the domain              [m]
rho = 1.0         # density                           [kg/m^3]
Gamma = 0.1       # diffusion coefficient             [kg/(m s)]

# ---- boundary conditions -------------------------------------------------
phi_0 = 1.0       # phi at x = 0 (left reservoir)
phi_L = 0.0       # phi at x = L (right reservoir)

# ---- the two flow velocities ---------------------------------------------
u_slow = 0.1      # case 1                            [m/s]
u_fast = 2.5      # case 2                            [m/s]
```

Problems 1 to 4 were about heat diffusing through a solid. In a **fluid** the material itself moves and **carries** ϕ with it: this is **convection**. Every equation of fluid flow, including the Navier–Stokes equations, has a convection term; it is what makes CFD *fluid* dynamics. Which effect wins is measured by the **Péclet number**:

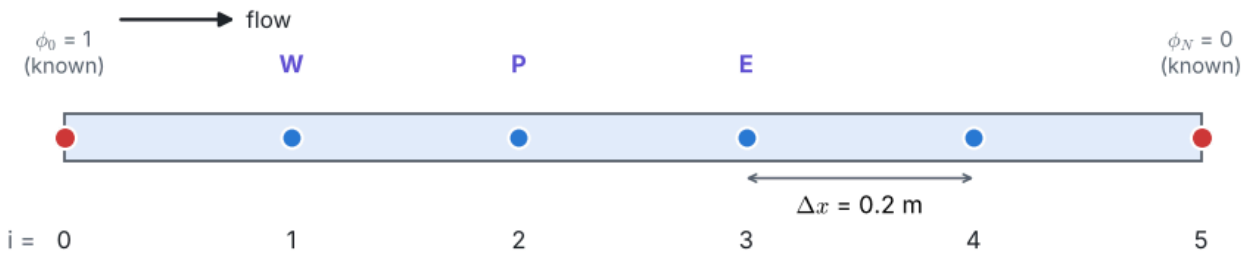
$$Pe_L = \frac{\text{convection}}{\text{diffusion}} = \frac{\rho u L}{\Gamma}.$$

```
for u in (u_slow, u_fast):
    print(f"u = {u:3.1f} m/s:   Pe_L = rho u L / Gamma = {rho * u * L / Gamma:4.1f}")
```

```
u = 0.1 m/s:   Pe_L = rho u L / Gamma =  1.0
u = 2.5 m/s:   Pe_L = rho u L / Gamma = 25.0
```

Step 2 · The grid

We keep $N+1$ equally spaced nodes, $x_i = i\,\Delta x$ with $\Delta x = L/N$. Nodes 0 and N sit in the reservoirs (known values). With the coarse grid $N = 5$, nodes 1 to 4 are the four unknowns.



The grid: 6 nodes, two known (red) and four unknown (blue). An interior node P has a west neighbour W (upstream) and an east neighbour E (downstream).

```
N = 5                    # number of intervals
dx = L / N               # grid spacing [m]

x = []                   # list with the position of every node
for i in range(N + 1):
    x.append(i * dx)

print(f"N = {N} intervals, dx = {dx:.1f} m, {len(x)} nodes")
print("node positions [m]:", [round(xi, 2) for xi in x])
```

```
N = 5 intervals, dx = 0.2 m, 6 nodes
node positions [m]: [0.0, 0.2, 0.4, 0.6, 0.8, 1.0]
```

On the grid, the Péclet number of **one interval**, the **cell Péclet number**, will decide how our schemes behave:

$$Pe_\Delta = \frac{\rho u \Delta x}{\Gamma}.$$

```
for u in (u_slow, u_fast):
    print(f"u = {u:3.1f} m/s:   cell Peclet number Pe = rho u dx / Gamma = {rho * u * dx /
Gamma:3.1f}")
```

```
u = 0.1 m/s:   cell Peclet number Pe = rho u dx / Gamma = 0.2
u = 2.5 m/s:   cell Peclet number Pe = rho u dx / Gamma = 5.0
```

Step 3 · Physics: the convection–diffusion equation

Through every cross-section flow two fluxes of ϕ: the **convective flux** $\rho u \phi$ (carried by the flow) and the **diffusive flux** $-\Gamma\, d\phi/dx$ (from high to low values). In steady state with no source the total flux is the same everywhere, so

$$\frac{d(\rho u \phi)}{dx} = \frac{d}{dx}\left(\Gamma \frac{d\phi}{dx}\right), \qquad \phi(0) = \phi_0 = 1, \qquad \phi(L) = \phi_L = 0.$$

Continuity, $d(\rho u)/dx = 0$, makes ρu constant. The two terms behave very differently:

- **diffusion** spreads ϕ in **both** directions;
- **convection** carries ϕ in **one** direction only, downstream. This is called **transportiveness**: the value at a point is mostly decided by what comes from upstream.

Exact solution (used to judge every numerical result):

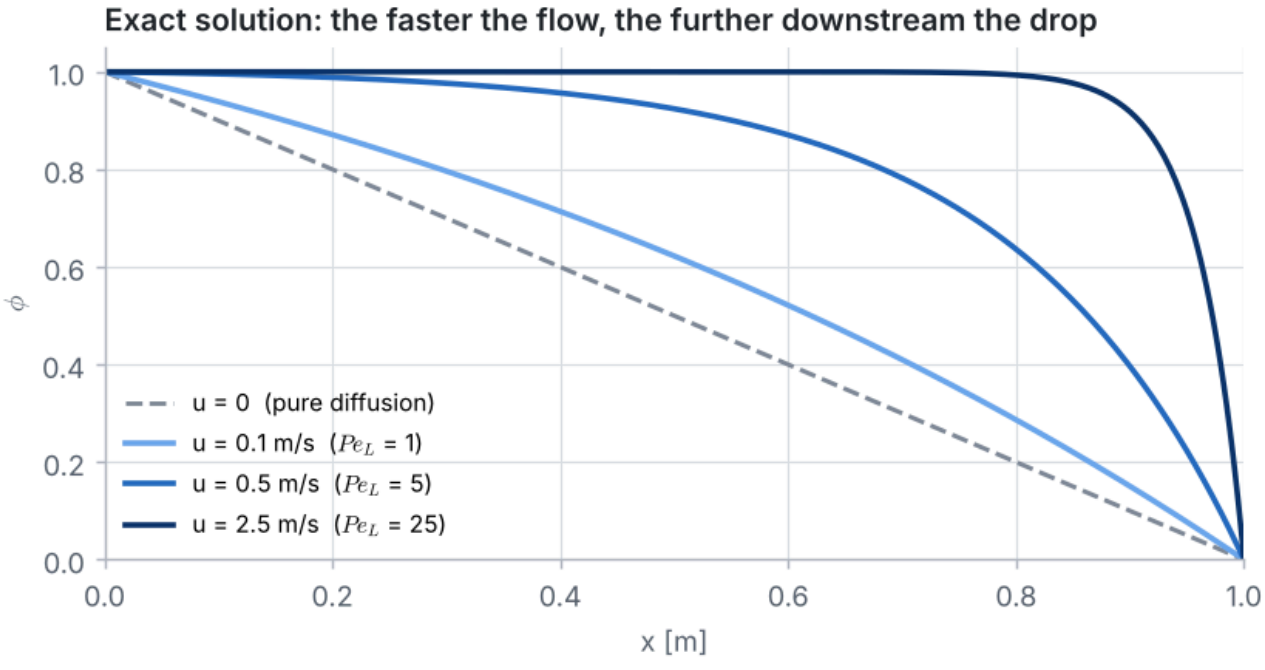
$$\frac{\phi(x) - \phi_0}{\phi_L - \phi_0} = \frac{\exp(\rho u x/\Gamma) - 1}{\exp(\rho u L/\Gamma) - 1}.$$

It always stays between 0 and 1: with no source, ϕ cannot exceed the largest boundary value or fall below the smallest. A good scheme must keep this property, called **boundedness**.

```
def phi_exact(xx, u):
    """Exact solution at position xx [m] for velocity u [m/s]."""
    Pe_L = rho * u * L / Gamma
    return phi_0 + (phi_L - phi_0) * (math.exp(Pe_L * xx / L) - 1) / (math.exp(Pe_L) - 1)


for u in (u_slow, u_fast):
    print(f"u = {u:3.1f} m/s:  exact phi at the nodes =", [round(phi_exact(xi, u), 4) for xi in
x])
```

```
u = 0.1 m/s:  exact phi at the nodes = [1.0, 0.8711, 0.7138, 0.5215, 0.2868, 0.0]
u = 2.5 m/s:  exact phi at the nodes = [1.0, 1.0, 1.0, 1.0, 0.9933, 0.0]
```



The faster the flow, the further downstream the drop. At $u = 2.5$ m/s, ϕ falls from 1 to 0 in a thin boundary layer, about $\Gamma/(\rho u) = 0.04$ m thick, in front of the outlet.

Step 4 · Numerical scheme: CDS and UDS

At an interior node i (P, with neighbours $W = i-1$ and $E = i+1$), the diffusion term gets the usual central difference, $\Gamma\,(\phi_{i-1} - 2\phi_i + \phi_{i+1})/\Delta x^2$. For the convection term we compare two choices:

$$\text{CDS (2nd order):}\quad \rho u\,\frac{\phi_{i+1} - \phi_{i-1}}{2\,\Delta x}, \qquad\qquad \text{UDS, } u>0 \text{ (1st order):}\quad \rho u\,\frac{\phi_i - \phi_{i-1}}{\Delta x}.$$

Upwind uses only the node the flow comes **from**, like the flow itself. Multiplying by Δx and introducing the flux strengths $F = \rho u$ and $D = \Gamma/\Delta x$ (so that $F/D = Pe_\Delta$), both schemes give

$$a_P\,\phi_i = a_W\,\phi_{i-1} + a_E\,\phi_{i+1}, \qquad a_P = a_W + a_E,$$

	a_W (west)	a_E (east)	a_P
central (CDS)	$D + F/2$	$D - F/2$	$2D$
upwind (UDS), $u>0$	$D + F$	D	$2D + F$

```python
def coefficients(u, N, scheme):
    """Neighbour coefficients a_W, a_E, a_P of an interior node."""
    F = rho * u               # convective flux strength [kg/(m^2 s)]
    D = Gamma / (L / N)       # diffusive flux strength  [kg/(m^2 s)]
    if scheme == "CDS":
        a_W = D + F / 2
        a_E = D - F / 2
    else:                     # "UDS", for a flow from left to right (u > 0)
        a_W = D + F
        a_E = D
    return a_W, a_E, a_W + a_E
```

```python
print("  u [m/s]  scheme    a_W     a_E     a_P    |a_W|+|a_E| <= a_P ?")
for u in (u_slow, u_fast):
    for scheme in ("CDS", "UDS"):
        a_W, a_E, a_P = coefficients(u, N, scheme)
        dominant = "yes" if abs(a_W) + abs(a_E) <= a_P + 1e-12 else "NO"
        print(f"   {u:4.1f}    {scheme}   {a_W:6.3f}  {a_E:6.3f}  {a_P:6.3f}        {dominant}")
```

```
  u [m/s]  scheme    a_W     a_E     a_P    |a_W|+|a_E| <= a_P ?
    0.1    CDS    0.550   0.450   1.000        yes
    0.1    UDS    0.600   0.500   1.100        yes
    2.5    CDS    1.750  -0.750   1.000        NO
    2.5    UDS    3.000   0.500   3.500        yes
```

With $Pe_\Delta = 5$, the CDS east coefficient is **negative** and the equation is no longer **diagonally dominant** ($|a_W| + |a_E| > a_P$). This happens whenever $F > 2D$, i.e. $Pe_\Delta > 2$. Upwind keeps all coefficients positive for any velocity.

As in Problem 1, every equation is stored as $a_i\,\phi_{i-1} + b_i\,\phi_i + c_i\,\phi_{i+1} = r_i$, a **tridiagonal** system, and the two boundary rows simply say $\phi_0 = 1$ and $\phi_N = 0$:

```python
def build_system(u, N, scheme):
    """Coefficients a, b, c, r of the FDM equations for N intervals."""
    a_W, a_E, a_P = coefficients(u, N, scheme)

    a = [0.0] * (N + 1)   # coefficient of phi[i-1]
    b = [0.0] * (N + 1)   # coefficient of phi[i]
    c = [0.0] * (N + 1)   # coefficient of phi[i+1]
    r = [0.0] * (N + 1)   # right-hand side

    # node 0, left reservoir: phi[0] = phi_0
    b[0] = 1.0
    r[0] = phi_0

    # interior nodes: -a_W phi[i-1] + a_P phi[i] - a_E phi[i+1] = 0
    for i in range(1, N):
        a[i] = -a_W
        b[i] = a_P
        c[i] = -a_E

    # node N, right reservoir: phi[N] = phi_L
    b[N] = 1.0
    r[N] = phi_L

    return a, b, c, r
```

For the fast flow with CDS:

```python
a, b, c, r = build_system(u_fast, N, "CDS")
for i in range(N + 1):
    terms = []
    if a[i] != 0:
        terms.append(f"{a[i]:+.2f}·phi{i - 1}")
    terms.append(f"{b[i]:+.2f}·phi{i}")
    if c[i] != 0:
        terms.append(f"{c[i]:+.2f}·phi{i + 1}")
    print(f"node {i}:  {'  '.join(terms):34s} = {r[i]:4.1f}")
```

```
node 0:  +1.00·phi0                          =  1.0
node 1:  -1.75·phi0  +1.00·phi1  +0.75·phi2 =  0.0
node 2:  -1.75·phi1  +1.00·phi2  +0.75·phi3 =  0.0
node 3:  -1.75·phi2  +1.00·phi3  +0.75·phi4 =  0.0
node 4:  -1.75·phi3  +1.00·phi4  +0.75·phi5 =  0.0
node 5:  +1.00·phi5                          =  0.0
```

Step 5 · Solution setup: the Thomas algorithm

The system is tridiagonal, so we solve it **directly** with the Thomas algorithm of Problem 1. It solves each scheme's equations exactly (up to round-off), whatever the signs of the coefficients. Any strange result must then come from the **scheme**, not from the solver.

```python
def thomas(a, b, c, r):
    """Solve a tridiagonal system directly (Thomas algorithm)."""
    n = len(b)
    c_star = [0.0] * n
    r_star = [0.0] * n

    # forward sweep: remove phi[i-1] from every equation
    c_star[0] = c[0] / b[0]
    r_star[0] = r[0] / b[0]
    for i in range(1, n):
        denominator = b[i] - a[i] * c_star[i - 1]
        c_star[i] = c[i] / denominator
        r_star[i] = (r[i] - a[i] * r_star[i - 1]) / denominator

    # back substitution: from the right end back to the left end
    phi = [0.0] * n
    phi[n - 1] = r_star[n - 1]
    for i in range(n - 2, -1, -1):
        phi[i] = r_star[i] - c_star[i] * phi[i + 1]

    return phi


def solve(u, N, scheme):
    """Build and solve the FDM equations: phi at the N+1 nodes."""
    return thomas(*build_system(u, N, scheme))
```

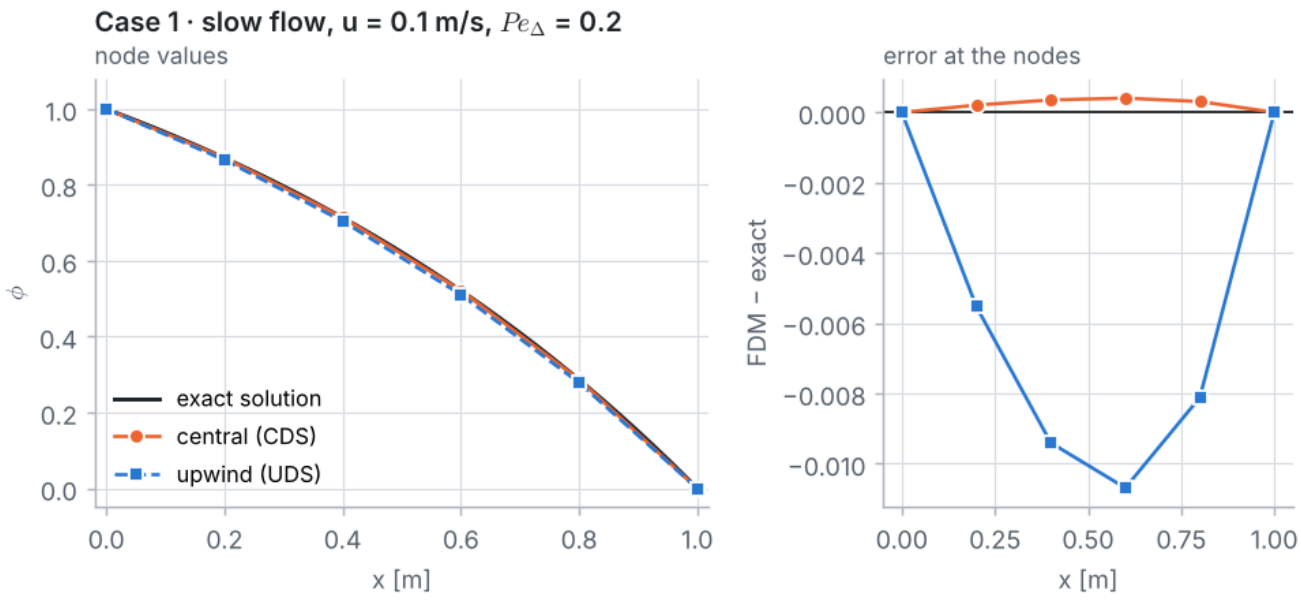
Step 6 · Compute

```python
def compare(u, N):
    """Print CDS, UDS and the exact solution node by node."""
    phi_cds = solve(u, N, "CDS")
    phi_uds = solve(u, N, "UDS")
    print(" node   x [m]     CDS       UDS      exact    error CDS  error UDS")
    for i in range(N + 1):
        exact = phi_exact(x[i], u)
        print(f"{i:4d}    {x[i]:4.1f}   {phi_cds[i]:8.4f}  {phi_uds[i]:8.4f}  {exact:8.4f}"
              f"   {phi_cds[i] - exact:+8.4f}   {phi_uds[i] - exact:+8.4f}")
    return phi_cds, phi_uds
```

Case 1 · slow flow, $u = 0.1$ m/s ($Pe_\Delta = 0.2$).

```python
phi_cds_slow, phi_uds_slow = compare(u_slow, N)
```

```
 node   x [m]     CDS       UDS      exact    error CDS  error UDS
   0     0.0    1.0000    1.0000    1.0000    +0.0000    +0.0000
   1     0.2    0.8714    0.8656    0.8711    +0.0002    -0.0055
   2     0.4    0.7141    0.7044    0.7138    +0.0004    -0.0094
   3     0.6    0.5220    0.5109    0.5215    +0.0004    -0.0107
   4     0.8    0.2871    0.2786    0.2868    +0.0003    -0.0081
   5     1.0    0.0000    0.0000    0.0000    +0.0000    +0.0000
```

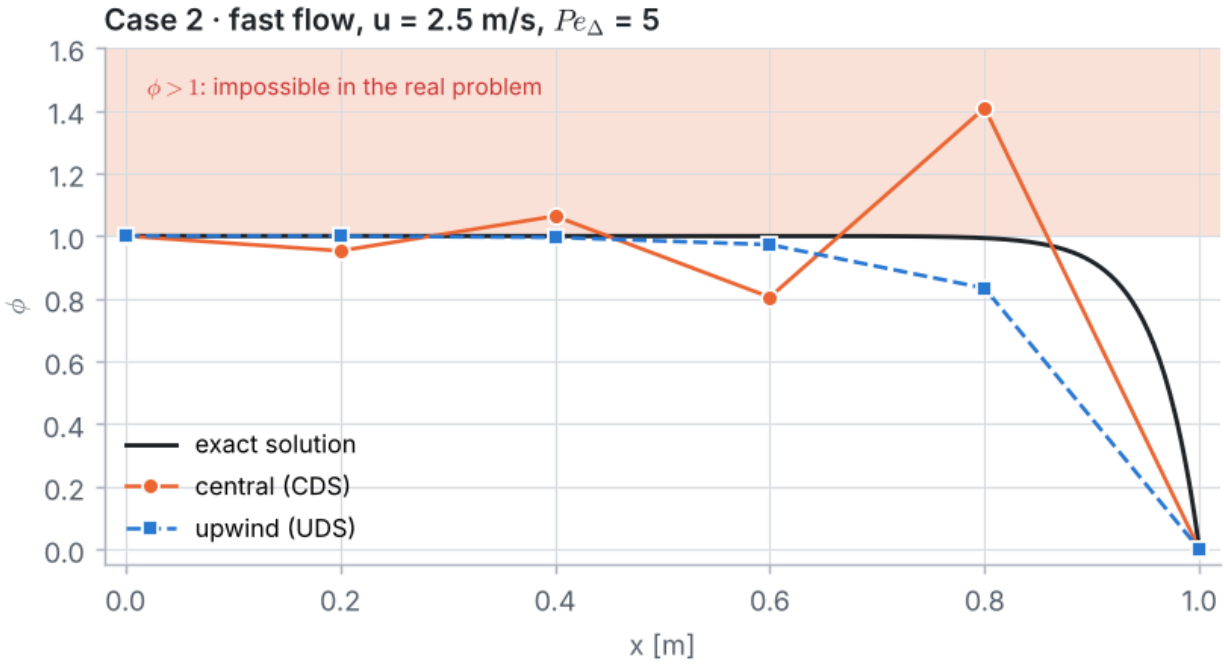


Case 1: both schemes follow the exact solution. The right panel shows the error at each node: CDS stays below 0.0005, UDS reaches 0.011.

Case 2 · fast flow, $u = 2.5$ m/s ($Pe_\Delta = 5$).

```
phi_cds_fast, phi_uds_fast = compare(u_fast, N)
```

```
 node   x [m]      CDS       UDS      exact    error CDS  error UDS
    0     0.0   1.0000    1.0000    1.0000    +0.0000    +0.0000
    1     0.2   0.9525    0.9994    1.0000    -0.0475    -0.0006
    2     0.4   1.0633    0.9955    1.0000    +0.0633    -0.0045
    3     0.6   0.8047    0.9723    1.0000    -0.1953    -0.0276
    4     0.8   1.4082    0.8334    0.9933    +0.4149    -0.1598
    5     1.0   0.0000    0.0000    0.0000    +0.0000    +0.0000
```



Case 2: CDS zig-zags and climbs to 1.41, a value that cannot exist in the real problem (shaded). UDS stays between 0 and 1 but smears the steep drop at the outlet.

> **CDS at $Pe_\Delta = 5$ is not just inaccurate, it is wrong.** The values go down, up, down, up (0.95, 1.06, 0.80, 1.41), and two of them exceed 1, the largest value present anywhere in the problem. The Thomas algorithm solved the CDS equations exactly: the wiggles are the true answer of the CDS equations on this grid. **UDS** is bounded and monotone, but at node 4 it gives 0.83 instead of 0.99.

The whiteboard guide solves both cases by hand with the Thomas algorithm, writing $\phi_i = P_i\,\phi_{i+1} + Q_i$, with all coefficients divided by D. The notebook prints the same intermediate numbers:

```
for u, scheme in ((u_slow, "CDS"), (u_fast, "CDS"), (u_fast, "UDS")):
    a_W, a_E, a_P = [coef / (Gamma / dx) for coef in coefficients(u, N, scheme)]   # divide by D
    print(f"u = {u} m/s, {scheme}:  a_W = {a_W:.2f}, a_P = {a_P:.2f}, a_E = {a_E:.2f}  (divided by
D)")
    P, Q = 0.0, phi_0                       # node 0: phi_0 = 0 * phi_1 + phi_0
    for i in range(1, N):
        denominator = a_P - a_W * P
        P, Q = a_E / denominator, a_W * Q / denominator
        print(f"   i = {i}:  a_P - a_W P = {denominator:8.6f}   P = {P:+.6f}   Q = {Q:.6f}")
```

```
u = 0.1 m/s, CDS:  a_W = 1.10, a_P = 2.00, a_E = 0.90  (divided by D)
   i = 1:  a_P - a_W P = 2.000000   P = +0.450000   Q = 0.550000
   i = 2:  a_P - a_W P = 1.505000   P = +0.598007   Q = 0.401993
   i = 3:  a_P - a_W P = 1.342193   P = +0.670545   Q = 0.329455
   i = 4:  a_P - a_W P = 1.262401   P = +0.712927   Q = 0.287073
u = 2.5 m/s, CDS:  a_W = 3.50, a_P = 2.00, a_E = -1.50  (divided by D)
   i = 1:  a_P - a_W P = 2.000000   P = -0.750000   Q = 1.750000
   i = 2:  a_P - a_W P = 4.625000   P = -0.324324   Q = 1.324324
   i = 3:  a_P - a_W P = 3.135135   P = -0.478448   Q = 1.478448
   i = 4:  a_P - a_W P = 3.674569   P = -0.408211   Q = 1.408211
u = 2.5 m/s, UDS:  a_W = 6.00, a_P = 7.00, a_E = 1.00  (divided by D)
   i = 1:  a_P - a_W P = 7.000000   P = +0.142857   Q = 0.857143
   i = 2:  a_P - a_W P = 6.142857   P = +0.162791   Q = 0.837209
   i = 3:  a_P - a_W P = 6.023256   P = +0.166023   Q = 0.833977
   i = 4:  a_P - a_W P = 6.003861   P = +0.166559   Q = 0.833441
```

Step 7 · Post-processing and verification

Why CDS wiggles. Divide the node equation by a_P:

$$\phi_i = \frac{a_W}{a_P}\,\phi_{i-1} + \frac{a_E}{a_P}\,\phi_{i+1}.$$

With two **positive** weights that add up to 1, every node value lies between its neighbours, and no value can leave the range $[0, 1]$ set by the boundaries. With CDS at $Pe_\Delta = 5$ the weights are 1.75 and -0.75, and this averaging is lost.

The equations even give the shape of the answer. Trying $\phi_i = s^i$ works for $s = 1$ and for $s = a_W/a_E$, so

$$\phi_i = 1 - \frac{s^i - 1}{s^N - 1}, \qquad s_{CDS} = \frac{1 + Pe_\Delta/2}{1 - Pe_\Delta/2}, \qquad s_{UDS} = 1 + Pe_\Delta.$$

The exact solution at the nodes is the same formula with s replaced by e^{Pe_Δ}. A good scheme has s close to e^{Pe_Δ}; a **negative** s makes s^i flip sign at every node: a zig-zag.

```
for u in (u_slow, u_fast):
    Pe = rho * u * dx / Gamma
    print(f"Pe = {Pe:3.1f}:   exact e^Pe = {math.exp(Pe):7.3f}    CDS s = {(1 + Pe / 2) / (1 - Pe
/ 2):+7.3f}"
          f"    UDS s = {1 + Pe:+7.3f}")
```

```
Pe = 0.2:   exact e^Pe =   1.221    CDS s =  +1.222    UDS s =  +1.200
Pe = 5.0:   exact e^Pe = 148.413    CDS s =  -2.333    UDS s =  +6.000
```

Rule of thumb. Central differences for convection are bounded only when $Pe_\Delta \le 2$: then $a_E = D - F/2 \ge 0$ and the matrix is diagonally dominant. Upwind has $s = 1 + Pe_\Delta > 0$ and can never wiggle, but $s = 6$ is far below $e^5 = 148$: the profile is smeared.

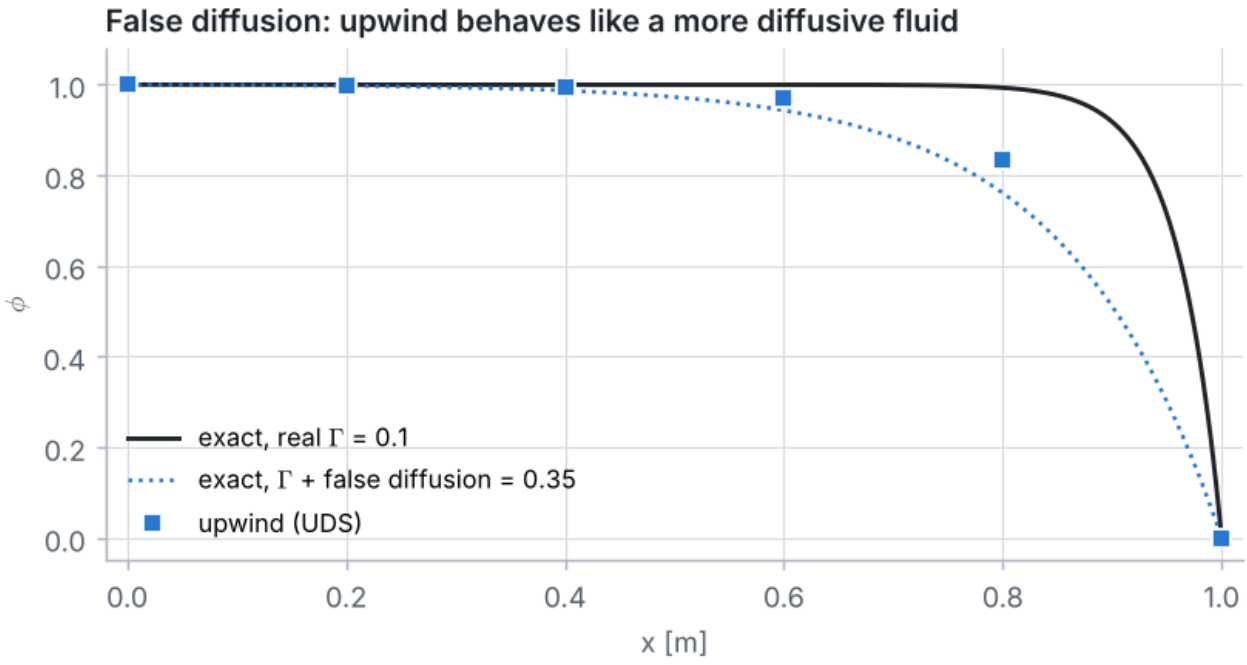
False diffusion. A backward difference is a central difference plus a diffusion-like correction:

$$\frac{\phi_i - \phi_{i-1}}{\Delta x} = \frac{\phi_{i+1} - \phi_{i-1}}{2\,\Delta x} - \frac{\Delta x}{2}\,\frac{\phi_{i+1} - 2\phi_i + \phi_{i-1}}{\Delta x^2}.$$

So the upwind equations are **exactly** the central equations with a larger diffusion coefficient, $\Gamma + \rho u\,\Delta x/2$. The extra part is called **false** (or **numerical**) **diffusion**.

```
for u in (u_slow, u_fast):
    Gamma_false = rho * u * dx / 2
    print(f"u = {u:3.1f} m/s:  false diffusion = {Gamma_false:.3f} kg/(m s)  "
          f"= {Gamma_false / Gamma:4.2f} x the real Gamma")
```

```
u = 0.1 m/s:  false diffusion = 0.010 kg/(m s)  = 0.10 x the real Gamma
u = 2.5 m/s:  false diffusion = 0.250 kg/(m s)  = 2.50 x the real Gamma
```



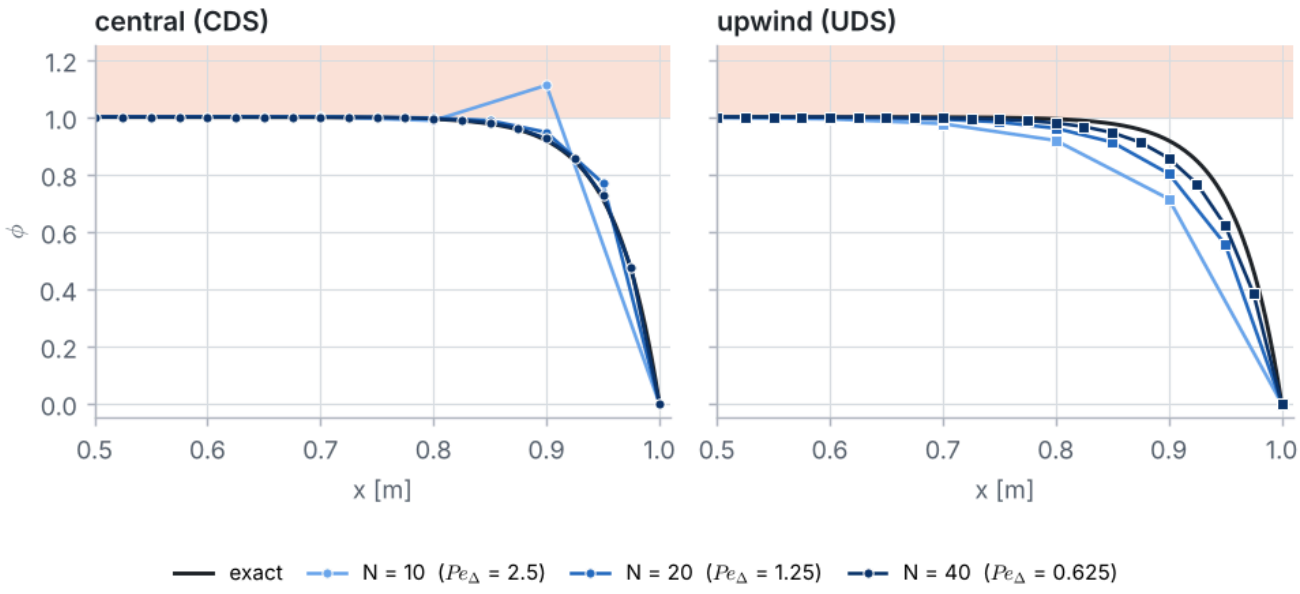
On the coarse grid, the upwind points drop early, like the exact solution of a fluid with $\Gamma = 0.35$ (dotted), instead of staying near 1 like the real fluid with $\Gamma = 0.1$ (solid). They do not lie exactly on the dotted curve: the upwind equations are the *central* equations for $\Gamma = 0.35$, which have their own discretization error.

Grid refinement (case 3). Both defects shrink with Δx: CDS becomes bounded once $Pe_\Delta < 2$ (for $u = 2.5$ m/s, $\Delta x < 0.08$ m), and false diffusion goes to zero. The observed order between two grids is

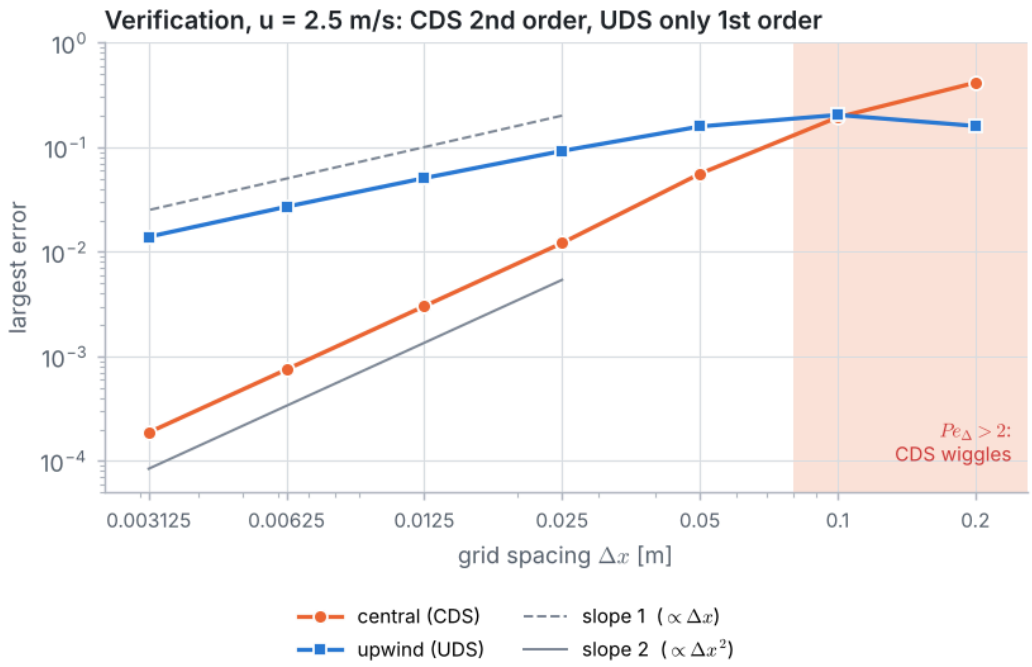
$$p = \log(e_{coarse}/e_{fine}) / \log 2.$$

```
grids = [5, 10, 20, 40, 80, 160, 320]
dxs, errors = [], {"CDS": [], "UDS": []}
print("    N    dx [m]     Pe     error CDS   order    error UDS   order")
for n_int in grids:
    line = f"{n_int:5d}  {L / n_int:8.6f}  {rho * u_fast * (L / n_int) / Gamma:5.3f}"
    for scheme in ("CDS", "UDS"):
        phi_n = solve(u_fast, n_int, scheme)
        error = max(abs(phi_n[i] - phi_exact(i * L / n_int, u_fast)) for i in range(n_int + 1))
        e = errors[scheme]
        order = f"{math.log(e[-1] / error) / math.log(2):5.2f}" if e else "     "
        e.append(error)
        line += f"   {error:9.6f}   {order}"
    dxs.append(L / n_int)
    print(line)
```

```
    N    dx [m]     Pe     error CDS   order    error UDS   order
    5  0.200000  5.000    0.414949            0.159822
   10  0.100000  2.500    0.193196    1.10    0.203627   -0.35
   20  0.050000  1.250    0.055736    1.79    0.157940    0.37
   40  0.025000  0.625    0.012128    2.20    0.092193    0.78
   80  0.012500  0.312    0.003021    2.01    0.050679    0.86
  160  0.006250  0.156    0.000749    2.01    0.026983    0.91
  320  0.003125  0.078    0.000187    2.00    0.013922    0.95
```



Zoom on the boundary layer. CDS overshoots on 10 intervals ($Pe_\Delta = 2.5$), has no overshoot on 20 and lies on the exact curve on 40. UDS approaches it only slowly.



Largest error against grid spacing. In the shaded region ($Pe_\Delta > 2$) CDS wiggles; once out of it, its error falls along a line of slope 2. UDS approaches slope 1.

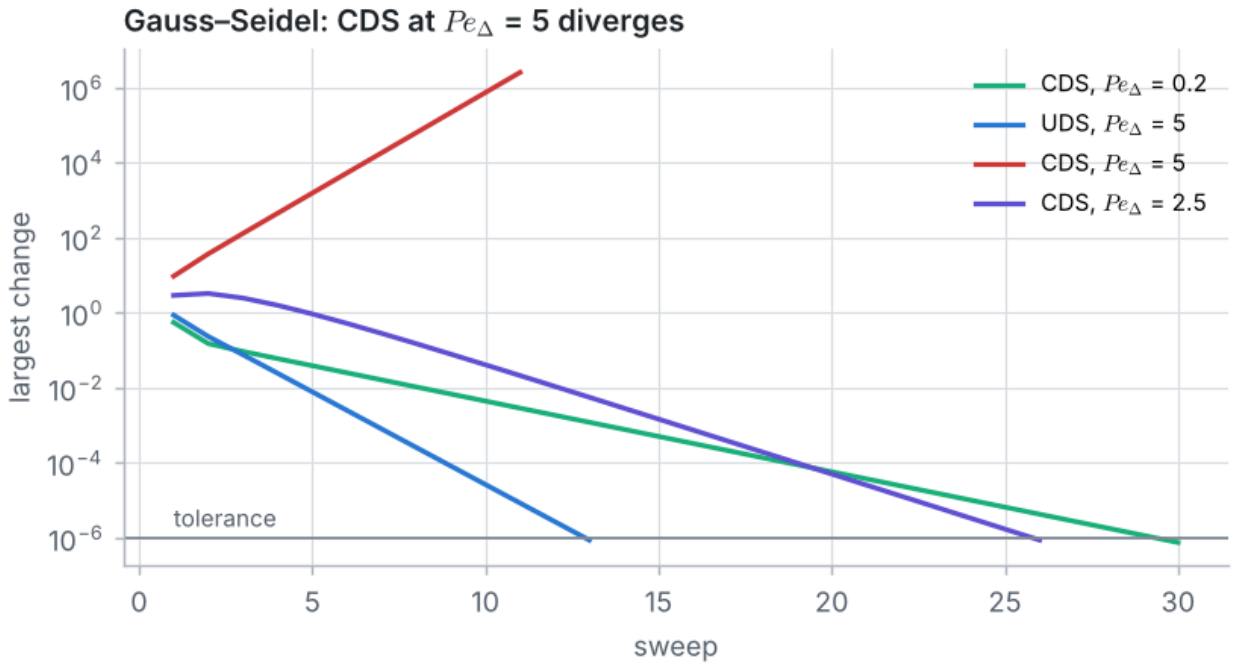
- **CDS** fails while $Pe_\Delta > 2$ ($N = 5$ and 10). From $N = 40$ on its error falls by a factor of 4 per halving of Δx: **2nd order**.
- **UDS** never wiggles, but its error even grows from $N = 5$ to $N = 10$ (on the coarsest grid no node lies inside the boundary layer), and on the finest grids it only halves per halving of Δx (observed order 0.95): **1st order**. With 320 intervals UDS (0.014) is still less accurate than CDS with 40 intervals (0.012).

Iterative solvers. Gauss–Seidel is guaranteed to converge for diagonally dominant systems. Starting from $\phi = 0$ inside the domain:

```python
def gauss_seidel(u, N, scheme, tol=1e-6, max_sweeps=200):
    """Gauss-Seidel sweeps for the interior nodes; returns the largest change of every sweep."""
    a_W, a_E, a_P = coefficients(u, N, scheme)
    phi = [phi_0] + [0.0] * (N - 1) + [phi_L]      # initial guess: 0 inside
    changes = []
    for sweep in range(max_sweeps):
        largest_change = 0.0
        for i in range(1, N):
            new_value = (a_W * phi[i - 1] + a_E * phi[i + 1]) / a_P
            largest_change = max(largest_change, abs(new_value - phi[i]))
            phi[i] = new_value                     # use the new value immediately
        changes.append(largest_change)
        if largest_change < tol or largest_change > 1e6:   # converged, or blown up
            break
    return changes


gs_runs = {}
for u, n_int, scheme in ((u_slow, 5, "CDS"), (u_fast, 5, "UDS"), (u_fast, 5, "CDS"), (u_fast, 10,
"CDS")):
    changes = gauss_seidel(u, n_int, scheme)
    Pe = rho * u * (L / n_int) / Gamma
    verdict = "converged" if changes[-1] < 1e-6 else "DIVERGED"
    gs_runs[(scheme, Pe)] = changes
    print(f"{scheme}, Pe = {Pe:3.1f}, N = {n_int:2d}:  {verdict} after {len(changes):3d} sweeps "
          f"(last change {changes[-1]:.1e})")
```

```
CDS, Pe = 0.2, N =  5:  converged after  30 sweeps (last change 7.5e-07)
UDS, Pe = 5.0, N =  5:  converged after  13 sweeps (last change 8.7e-07)
CDS, Pe = 5.0, N =  5:  DIVERGED after  11 sweeps (last change 2.5e+06)
CDS, Pe = 2.5, N = 10:  converged after  26 sweeps (last change 8.7e-07)
```



Largest change per Gauss–Seidel sweep. CDS at $Pe_\Delta = 5$ explodes; the other three runs reach the tolerance.

CDS at $Pe_\Delta = 5$ **diverges**. CDS at $Pe_\Delta = 2.5$ is not diagonally dominant either, yet it converges: losing diagonal dominance removes the *guarantee*, it does not always mean divergence.

Why commercial codes do not use plain central differences for convection. Real flows have very large Péclet numbers, so keeping $Pe_\Delta < 2$ everywhere would need far too many cells, while plain upwinding

is too diffusive. Commercial codes therefore use **upwind-biased** schemes (second-order upwind, QUICK, TVD schemes with limiters) or add a controlled amount of numerical diffusion only where needed (**stabilisation**; COMSOL, for example, applies streamline-diffusion stabilisation by default in its transport interfaces).

> **Result.** For the slow flow ($Pe_\Delta = 0.2$) both schemes work on 5 intervals: the largest CDS error is 0.0004 and the largest UDS error 0.011. For the fast flow ($Pe_\Delta = 5$) CDS gives wiggles outside the physical range (up to 1.41 at node 4, exact 0.99), while UDS is bounded but smeared (0.83 at node 4) by false diffusion 2.5 times larger than the real diffusion. Refining the grid, CDS converges at **2nd order** once $Pe_\Delta < 2$, UDS at only **1st order**.

Key ideas

- The convection term carries ϕ downstream (transportiveness); the Péclet number compares it with diffusion.
- On a grid, the cell Péclet number $Pe_\Delta = \rho u\,\Delta x/\Gamma$ decides how a scheme behaves.
- Central differences are 2nd order but unbounded for $Pe_\Delta > 2$: a negative neighbour coefficient, loss of diagonal dominance, wiggles, and Gauss–Seidel may diverge.
- Upwind differences are always bounded but only 1st order, and add false diffusion $\rho u\,\Delta x/2$.
- Practical CFD codes use upwind-biased or stabilised schemes to get both boundedness and accuracy.